

Paths, Rate Points, and the Class C Decomposition

A reference note on MAC polar code structure

2026-04-09

1. The MAC capacity region

For a two-user multiple access channel with inputs X (user U) and Y (user V) and output Z , the capacity region is determined by three mutual information constraints:

- $R_u \leq I(X; Z | Y)$ — user U 's rate, given V is known
- $R_v \leq I(Y; Z | X)$ — user V 's rate, given U is known
- $R_u + R_v \leq I(X, Y; Z)$ — sum of both users' rates

Taking GMAC at SNR = 6 dB (from `designs/gmac_B_n8_snr6dB.npz`) as a concrete example:

- $I(X; Z | Y) = 0.9119$ (clean BPSK+AWGN for X given Y is known)
- $I(Y; Z | X) = 0.9119$ (symmetric)
- $I(X, Y; Z) = 1.3764$ (joint)

The region is a pentagon, and the interesting edge is the **dominant face** — the line segment connecting the two corners where both the individual and the sum constraints are active:

- Corner A: $(R_u, R_v) = (I(X; Z), I(Y; Z | X)) = (0.4645, 0.9119)$
- Corner B: $(R_u, R_v) = (I(X; Z | Y), I(Y; Z)) = (0.9119, 0.4645)$

Both corners satisfy $R_u + R_v = I(X, Y; Z)$. Every point between them on this line (parameterized by a convex combination) is achievable.

2. Paths and lattice walks

Polar codes for the MAC use a monotone **decoding order**, called a path, represented as a $2N$ -bit sequence $\mathbf{b} = (b_1, b_2, \dots, b_{2N})$. Each symbol tells the joint decoder “the next bit I’m decoding belongs to user U or user V ”:

- $b_t = 0$ — decode a U -bit at step t
- $b_t = 1$ — decode a V -bit at step t

The path must have exactly N zeros and N ones (decode N bits per user).

A path can be visualized as a **monotone lattice walk** from $(0, 0)$ to (N, N) on an integer grid. At each step, the “state” (k_u, k_v) records how many U and V bits have been decoded so far. A U -step moves right; a V -step moves up. Different paths trace different routes from origin to destination, and each route visits a different sequence of side-information states.

There are $C(2N, N)$ such lattice paths. For $N = 4$, that is 70 paths. For $N = 256$, that is an astronomical number.

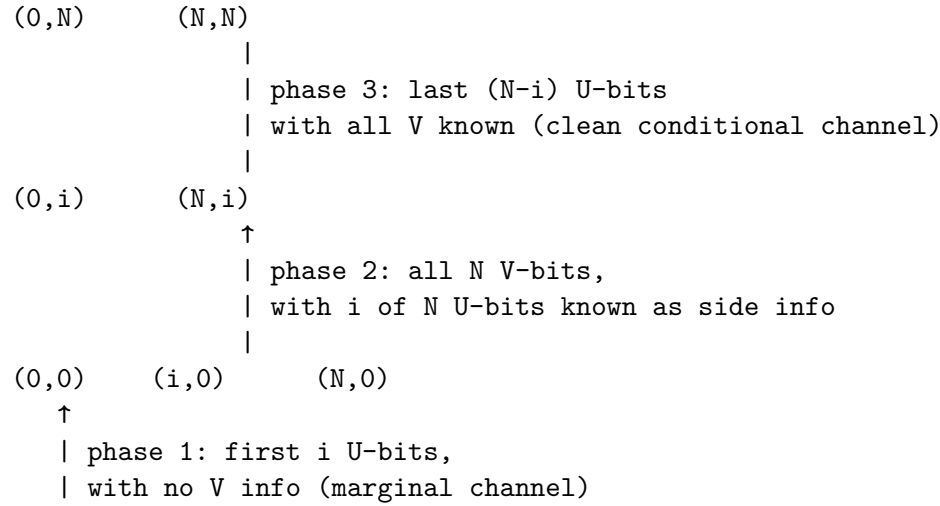
3. The family $0^i 1^N 0^{N-i}$

One simple and important family of monotone paths consists of “step” paths:

$$\text{path}(i) = 0^i 1^N 0^{N-i}$$

Reading left to right: decode i bits of U first, then decode all N bits of V , then decode the remaining $N - i$ bits of U . As i ranges over $\{0, 1, \dots, N\}$, this family has $N + 1$ paths.

The lattice walk for $\text{path}(i)$ has a characteristic staircase shape:



The two extreme values of i correspond to the corners of the dominant face:

- $i = 0$: path $1^N 0^N$, decode all V first and then all U . V sees the marginal channel, U sees the clean conditional channel.
- $i = N$: path $0^N 1^N$, decode all U first and then all V . U sees the marginal channel, V sees the clean conditional channel.

Intermediate values of i trace a curve between these two corners. Finite- N data (Section 5) confirms the curve is monotone in i but is **not** a simple linear interpolation.

4. A wrong formula and how it was corrected

During early discussion I proposed the asymptotic formula

$$i / N = (R_u - I(X; Z)) / (I(X; Z | Y) - I(X; Z))$$

which would give a clean linear interpolation of (R_u, R_v) as i varies. **This formula is wrong.** I verified it against real design files at $N = 256$ and got large discrepancies:

	k_u / N (actual)	k_v / N (actual)	k_u / N (my formula)	k_v / N (my formula)
Class B ($i = 128$)	0.5273	0.5977	0.6882	0.6882

	k_u / N (actual)	k_v / N (actual)	k_u / N (my formula)	k_v / N (my formula)
Class C ($i = 256$)	0.3164	0.8281	0.9119	0.4645

Two distinct errors:

Error 1. The Class C corner direction was flipped. Class C is $0^N 1^N$, meaning all U first (no V info), so U sees the **marginal** channel and R_u is small. My formula predicted $R_u = 0.9119$; the correct corner is $(R_u, R_v) = (0.4645, 0.9119)$, and the finite-N achievable rate (0.3164, 0.8281) matches that pattern with room left for margin below capacity.

Error 2. The path $0^{\{N/2\}} 1^N 0^{\{N/2\}}$ is **not symmetric under user swap**. V is decoded in a single block of N bits (with half of U known), while U is decoded in two blocks (one with zero V info and one with full V info). These are structurally different decoding regimes, so U’s achieved rate differs from V’s achieved rate. The data shows U’s rate is lower than V’s at $i = N/2$.

The correct honest answer. There is no simple closed-form function from i to (R_u, R_v) . Asymptotically, each path achieves some point on the dominant face; computing exactly where requires bookkeeping Önay’s chain-rule mutual information terms along the lattice walk. At finite N, the only reliable method is **Monte Carlo density evolution**: simulate the channel many times, estimate per-position error rates, and count how many positions are reliable. This is exactly what the .npz design files in the project store.

5. What “Class A/B/C” actually mean in the project

The code in polar/design.py defines:

```
def make_path(N: int, path_i: int) -> list:
    """Build path  $b = 0^{\{path\_i\}} 1^N 0^{\{N - path\_i\}}$ ."""
    return [0] * path_i + [1] * N + [0] * (N - path_i)
```

and the simulation scripts hardcode three path_i_frac values:

Class	path_i_frac	path_i at N = 256
A	0.375	96
B	0.500	128
C	1.000	256

The comments in simulate.py say the A and B values were picked “from sweep” — meaning somebody empirically tried different path_i values for a target rate point and saved the ones that worked best. **These path_i values are frozen across all channels** (BEMAC, ABNMAC, GMAC at all SNRs).

The consequence: because the path is frozen but the channel changes, **the achieved (R_u, R_v) rate pair is different for every channel**. The same path_i = 128 gives:

Channel	k_u / N	k_v / N	Sum
BEMAC	0.559	0.781	1.340
ABNMAC	0.457	0.434	0.891
GMAC @ 3 dB	0.426	0.398	0.824
GMAC @ 6 dB	0.527	0.598	1.125
GMAC @ 10 dB	0.559	0.762	1.320

So “Class B” in this project is **not** a statement about rate point — it is just a specific lattice walk. The actual rate pair depends on the channel.

Only **Class C** is **theoretically principled**: it sits at the literal corner $i = N$ of the capacity region and has a clean two-phase structural interpretation. Class A and Class B are arbitrary interior operating points selected empirically for the BEMAC case.

6. The Class C decomposition

Because Class C is at the corner $i = N = \text{path } 0^N 1^N$, its decoding structure is exceptionally clean. No V-bit is decoded before any U-bit; no U-bit is decoded after any V-bit. The $2N$ decoding steps split cleanly into two consecutive phases of N steps each:

Phase 1 — decode U (no V info). Every U-step happens with $k_v = 0$. The effective channel seen by U’s polar code is the **marginal channel**

$$W_X(z \mid x) = \sum_y p(z \mid x, y) p(y) = (1/2)[p(z \mid x, y=0) + p(z \mid x, y=1)]$$

For GMAC with $Z = (1 - 2X) + (1 - 2Y) + W, W \sim N(0, \sigma^2)$:

$$\begin{aligned} p(z \mid x = 0) &= (1/2) [N(z; +2, \sigma^2) + N(z; 0, \sigma^2)] & (Y = 0 \text{ gives mean } +2, Y = 1 \text{ gives mean } 0) \\ p(z \mid x = 1) &= (1/2) [N(z; 0, \sigma^2) + N(z; -2, \sigma^2)] & (Y = 0 \text{ gives mean } 0, Y = 1 \text{ gives mean } -2) \end{aligned}$$

This is a **Gaussian mixture channel**. It is a standard single-user binary-input channel, just non-Gaussian. A single-user polar SC decoder handles it via a single modification: compute the leaf LLR using the mixture formula instead of the plain Gaussian formula.

Phase 2 — decode V given U. After Phase 1 produces $\hat{U}$, reconstruct $\hat{X} = \text{polar_encode}(\hat{U})$. Subtract the known contribution:

$$Z' = Z - (1 - 2\hat{X}) = (1 - 2Y) + W$$

This is clean BPSK + AWGN — a standard single-user channel. A standard single-user polar SC decoder decodes V from Z' .

Key observation. Both phases are **single-user polar decoding problems**. The 2-user MAC-specific structure — `CalcLeft`, `CalcRight`, `CalcParent` on 2×2 joint probability tensors — is not needed for Class C. Two single-user SC decoders chained in sequence suffice.

The project already implements this analytically. From `polar/decoder.py`:

```
def _decode_extreme_llr(N, log_W, path_i, frozen_u, frozen_v):
    """O(N log N) LLR-based SC for extreme paths (path_i = 0 or path_i = N)."""
    from .encoder import polar_encode
```

```

if path_i == N:      # Class C: decode all U first, then all V
    u_hat = _sc_decode_from_llr(_u_marginal_llr(log_W), frozen_u)
    x_hat = np.array(polar_encode(u_hat.tolist()), dtype=np.int8)
    v_hat = _sc_decode_from_llr(_v_conditional_llr(log_W, x_hat), frozen_v)
    return u_hat.tolist(), v_hat.tolist()
else:                # path_i == 0: decode all V first, then all U
    v_hat = _sc_decode_from_llr(_v_marginal_llr(log_W), frozen_v)
    y_hat = np.array(polar_encode(v_hat.tolist()), dtype=np.int8)
    u_hat = _sc_decode_from_llr(_u_conditional_llr(log_W, y_hat), frozen_u)
    return u_hat.tolist(), v_hat.tolist()

```

The project’s header comment even notes: `path_i=0` or `path_i=N` $\rightarrow O(N \log N)$ LLR-based SC decoder (faster, $\sim 2\times$). Extreme paths get a dedicated fast code path because the two-phase decomposition saves the overhead of maintaining joint tensors.

Important subtlety. “Marginalize over Y to get $Z | X$ ” does not mean pre-processing the channel output z into a different signal. You don’t integrate out Y before feeding z to the decoder. You use the same z as input, but compute leaf LLRs using the mixture formula. The marginalization happens inside the LLR computation, not in the data.

7. Why this matters for neural decoders

The 2-user MAC neural decoder in this project uses a 4-class joint (u, v) output at each leaf and maintains (2×2) joint probability tensors through the tree operations. This architecture has been shown in this session to have limitations at large N : even the production CG decoder achieves only BLER ~ 0.017 at $N = 256$ for Class B, which is about $3.3\times$ the analytical SC reference.

For Class C specifically, the 4-class joint architecture is **overkill**. The Class C decoder can be implemented as two chained single-user decoders, each with a **binary** decision at each leaf. This has several advantages:

1. **Reuse of single-user infrastructure.** Single-user polar decoders, neural or analytical, are a mature and well-studied object. Any working single-user NPD plugs directly into the two-stage Class C pipeline.
2. **No 4-class exposure bias.** The single-user NPD has a sign-flip symmetry at the BitNode that makes it robust to its own prediction errors. The 4-class MAC version has no equivalent symmetry and suffers from catastrophic exposure bias.
3. **Two simpler tasks instead of one hard one.** Phase 1 (decode U over the mixture channel) and Phase 2 (decode V over clean BPSK + AWGN) are each a standard single-user polar decoding problem over a specific channel. Phase 2 is the textbook Aharoni et al. NPD problem. Phase 1 is the same setup with a non-Gaussian channel.
4. **Generalizes across channels.** BEMAC, ABNMAC, GMAC, and even channels with memory (ISI-MAC) all admit the same decomposition. Only the Phase 1 mixture channel changes; Phase 2 is always the clean channel of the second user given known first-user side information.

The practical cost is that Class C is at a specific corner of the capacity region — highly asymmetric rates ($R_u \sim I(X;Z)$, $R_v \sim I(Y;Z|X)$). For GMAC this is about $(0.46, 0.91)$ rather than

symmetric (0.69, 0.69). If a symmetric rate point is required, Class C is not the right operating point.

8. Summary

1. Each monotone lattice path from $(0, 0)$ to (N, N) gives a distinct polar code. There is a one-parameter family $0 \leq i \leq N$ indexed by $i \in \{0, \dots, N\}$.
2. Different i values give different rate pairs on the dominant face. There is no simple closed-form formula from i to (R_u, R_v) ; the correct procedure is Monte Carlo density evolution.
3. The labels Class A, B, C in this project do **not** correspond to meaningful theoretical points in general. They are three empirically chosen values $\text{path}_i \in \{0.375N, 0.5N, N\}$, hardcoded across channels.
4. **Class C at $i = N$ is the exception:** it is a literal capacity-region corner with a clean two-phase decoding structure.
5. For Class C, decoding decomposes exactly into **two chained single-user SC decoders**: first decode U on the marginal channel, then reconstruct $\mathbf{X}$, then decode V on the clean conditional channel. The project already implements this analytically.
6. For the neural case, Class C offers the opportunity to reuse working single-user NPD components instead of the more delicate 4-class joint architecture. This is the foundation for a “Class C MAC NPD” project that applies the Aharoni et al. single-user NPD framework to multi-user channels — a natural next step that inherits the single-user method’s proven behavior and avoids the scalability issues of the joint 4-class decoder.